

Multi-Business Management Platform

A comprehensive Next.js 15 TypeScript application for managing multiple business operations including personal finance, construction projects, restaurant management, grocery inventory, clothing retail, and hardware operations. Features unified project management, contractor payments, expense tracking, user permissions, audit logging, and cross-business financial integration with support for loans, budgets, and real-time transaction processing..

Features

- **Multi-Business Support:** Construction, Restaurant, Grocery, Clothing, Personal Finance modules
- **Role-Based Access Control:** Granular permissions per business module
- **POS Systems:** Touch-friendly interfaces for retail businesses
- **Real-time Chat:** Internal messaging between team members
- **Comprehensive Reporting:** Analytics and financial reports
- **Audit Logging:** Complete activity tracking
- **Backup & Restore:** Automated data protection
- **Mobile Responsive:** Optimized for desktop, tablet, and mobile
- **Windows Service:** Self-hosted solution with automatic migrations
- **Peer-to-Peer Sync:** Multi-server database synchronization with mDNS discovery

Quick Start

Fresh Installation

```
# 1. Clone repository
git clone <repository-url>
cd multi-business-multi-apps

# 2. Configure environment
cp .env.example .env.local
# Edit .env.local with your settings

# 3. Run automated setup
npm run setup

# 4. For Production: Install Windows service (as Administrator)
npm run service:install

# 5. For Production: Start service (as Administrator)
npm run service:start

# OR for Development: Start dev server (no admin required)
npm run dev
```

What `npm run setup` does automatically:

- Installs dependencies
- Creates database if it doesn't exist
- Runs all migrations
- Seeds reference data (~128 records)
- Seeds business categories (20 categories, 59 subcategories)
- Builds application and service
- Creates admin user: `admin@business.local` / `admin123`

Development

```
npm run dev
# Access at http://localhost:8080
```

Production Updates

```
git pull
npm run setup:update
npm run service:restart # As Administrator
```

 **For complete deployment instructions, see [ADMIN-INSTALLATION-GUIDE.md](#)**

Windows Service Commands

All commands require Administrator privileges

```
# Service management
npm run service:install    # Install Windows service
npm run service:start      # Start service
npm run service:stop       # Stop service
npm run service:restart    # Restart service
npm run service:status     # Check status
npm run service:diagnose   # Diagnose issues
npm run service:uninstall  # Uninstall service
```

 **For troubleshooting and advanced service management, see [ADMIN-INSTALLATION-GUIDE.md](#)**

Project Structure

```
src/
├─ app/           # Next.js app router pages
├─ components/    # Reusable React components
├─ lib/           # Core utilities and configurations
```

```
| types/           # TypeScript type definitions
| modules/        # Business-specific modules
```

Environment Configuration

Create `.env.local` in the project root with these required variables:

```
# Database
DATABASE_URL="postgresql://postgres:password@localhost:5432/multi_business_db"

# Authentication
NEXTAUTH_URL="http://localhost:8080"
NEXTAUTH_SECRET="your-secret-key-here"

# Application
PORT=8080

# Sync Service (for multi-server deployments)
SYNC_NODE_ID="unique-per-server"
SYNC_NODE_NAME="sync-node-server1"
SYNC_REGISTRATION_KEY="same-on-all-servers"
SYNC_SERVICE_PORT=8765
```

Generate secure values:

Git Bash / Linux / macOS:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))" #
NEXTAUTH_SECRET
node -e "console.log(require('crypto').randomBytes(8).toString('hex'))"    #
SYNC_NODE_ID
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"    #
SYNC_REGISTRATION_KEY
```

PowerShell:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))" #
NEXTAUTH_SECRET
node -e "console.log(require('crypto').randomBytes(8).toString('hex'))"    #
SYNC_NODE_ID
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"    #
SYNC_REGISTRATION_KEY
```

Command Prompt (CMD):

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
node -e "console.log(require('crypto').randomBytes(8).toString('hex'))"
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

 See [ADMIN-INSTALLATION-GUIDE.md](#) for complete environment configuration guide

Available Scripts

Setup & Deployment

- `npm run setup` - Fresh installation (automated)
- `npm run setup:update` - Update after git pull
- `npm run hooks:install` - Install git hooks for automatic updates

Development

- `npm run dev` - Start development server (port 8080)
- `npm run build` - Build for production
- `npm run test` - Run test suite
- `npm run lint` - Run ESLint
- `npm run type-check` - Run TypeScript checks

Service Management (Administrator required)

- `npm run service:install` - Install as Windows service
- `npm run service:start` - Start service
- `npm run service:stop` - Stop service
- `npm run service:restart` - Restart service
- `npm run service:status` - Check service status
- `npm run service:diagnose` - Diagnose issues
- `npm run service:uninstall` - Remove Windows service

Database

- `npm run db:studio` - Open Prisma Studio GUI
- `npm run db:generate` - Generate Prisma client
- `npm run db:migrate` - Create migration (development)
- `npm run db:deploy` - Apply migrations (production)

 For complete command reference, see [ADMIN-INSTALLATION-GUIDE.md](#)

Module Overview

Construction Management

- Project tracking and budgeting
- Expense categorization with receipt attachments
- Progress reporting and analytics

Restaurant POS

- Touch-friendly point of sale interface
- Menu management and table service
- Order processing and kitchen communication

Grocery Shop

- Barcode scanning and inventory management
- Supplier tracking and purchase orders
- Expiry date monitoring and alerts

Clothing Outlet

- Size/color variant management
- Customer loyalty and discount systems
- Seasonal inventory tracking

Personal Finance

- Expense categorization and tracking
- Monthly spending reports and budgets
- Transfer tracking between business and personal accounts

Security Features

- JWT-based authentication
- Role-based access control (RBAC)
- Audit logging for all critical actions
- Encrypted backup files
- Session management
- Peer-to-peer sync with registration key authentication

Documentation

Deployment & Operations

- [ADMIN-INSTALLATION-GUIDE.md](#) - Complete deployment guide
 - Fresh installation workflow
 - Production updates
 - Windows service management
 - Environment configuration
 - Troubleshooting
 - Maintenance procedures
- [PRISMA_GUIDE.md](#) - Prisma best practices
 - Naming conventions
 - Safe vs dangerous commands

- Migration workflows
- Common issues & solutions

Project Planning

- [projectplan.md](#) - Current project planning and implementation notes

Getting Help

- For deployment issues, see [ADMIN-INSTALLATION-GUIDE.md - Troubleshooting](#)
- For Prisma/database issues, see [PRISMA_GUIDE.md](#)
- Check service logs: [logs/service.log](#) and [windows-service/daemon/service.log](#)

License

ISC